

FinInsights

FinInsights is a KPI intelligence-to-action engine for a bank. It watches banking activity, checks whether a movement is real and not a data glitch, finds the part of the business that caused it, says how confident it is, explains it in plain language for the person asking, and recommends an action with a named owner.

The data comes from NexaBank, a working retail bank that ships with this project. NexaBank has customers, accounts, transactions, loans with their KYC steps, cards and branches. It also has a Simulate console that plants anomalies in the source data, so the engine has to discover them rather than being handed them. Every number a reader sees traces back to a value that a tool computed and stored. The language model reads the question and writes the sentences. It never produces a number.

- Project page: https://github.com/abhishekkumawat-47/FinInsights-Business_Intelligence
- Issue queue: https://github.com/abhishekkumawat-47/FinInsights-Business_Intelligence/issues

Table of contents

- [Requirements](#)
- [Installation](#)
- [Configuration](#)
- [Usage](#)
- [Five minute demo](#)
- [Services and ports](#)
- [Troubleshooting and FAQ](#)
- [Working on the code](#)
- [Maintainers](#)

Requirements

Everything except Ollama runs inside Docker, so you do not install Python, Node or a database on your machine.

- Docker Desktop. Download it from <https://www.docker.com/products/docker-desktop/> and start it. Wait until it reports "Engine running". Nothing below works until it does.
- Ollama. Download it from <https://ollama.com/download>. This runs the small language model on your own machine, so the bank data never leaves it.
- About 10 GB of free disk space for the Docker images and the warehouse.

Ollama is installed normally instead of in Docker on purpose. The containers reach it through the address `host.docker.internal`, which is how a container talks to a program running on your machine.

Installation

Follow these steps in order. Every command can be copied and pasted.

1. Install Ollama and build the narrator model

Download the installer for your operating system from <https://ollama.com/download> and run it. On Windows it installs as a normal application and starts on its own. On macOS, drag it to Applications and open it once.

Check that it is running:

```
ollama --version
curl http://localhost:11434/api/tags
```

The second command should return JSON. A connection error means Ollama is not running yet.

Pull the base model. This is about 1.9 GB and only happens once:

```
ollama pull qwen2.5:3b-instruct
```

Now build the narrator model. This step is required, not optional. The default context length in Ollama is 2,048 tokens, and a longer prompt is cut short silently instead of being refused. The narrator sends about 4,500 tokens of evidence before it writes a word, so on the default setting it would write from a set of facts with the middle missing, and no error would appear anywhere.

On macOS, Linux or Git Bash:

```
cat > Modelfile <<'EOF'
FROM qwen2.5:3b-instruct
PARAMETER num_ctx 8192
EOF
ollama create fininsights-narrator -f Modelfile
```

On Windows PowerShell:

```
"FROM qwen2.5:3b-instruct`nPARAMETER num_ctx 8192" | Out-File -Encoding utf8 Modelfile
ollama create fininsights-narrator -f Modelfile
```

Confirm the model exists. `fininsights-narrator` should be in the list:

```
ollama list
```

2. Get the code

```
git clone https://github.com/abhishekkumawat-47/FinInsights-Business_Intelligence.git
cd FinInsights-Business_Intelligence
cp .env.example .env
```

You do not need to edit `.env` to run the demo. See [Configuration](#) for what the defaults mean.

3. Start the stack

```
docker compose up -d
```

The first run downloads images and builds the services, which takes about 5 to 15 minutes on a normal connection. After that it starts in under a minute.

Watch the services come up and wait until each one reads `running`:

```
docker compose ps
```

Two things are worth knowing here:

- Do not add `--profile gpu`. That starts an older vLLM server that this setup no longer uses. It will either fail or take GPU memory for nothing.
- The `migrate` container stops on purpose. It applies the warehouse schema and exits. Exit code 0 means it worked.

Check that the core services answer:

```
curl localhost:8001/health
curl localhost:8000/health
```

4. Load the data

The bank starts empty. This command generates several months of realistic banking activity and plants the demo anomalies in the source data:

```
docker compose --profile tools run --rm tools python scripts/seed_data.py --scenario all
```

This takes a few minutes and prints what it planted. It also writes `fixtures/planted_truth.json`, which records the ground truth so you can check later whether the engine found what was actually put there.

Build the warehouse layers. This reads the bank database, cleans the data and computes the KPIs:

```
curl -X POST localhost:8003/refresh
```

Score the findings. Each range is scored on its own, because a 7 day finding and a 90 day finding are different claims about different periods:

```
curl -X POST "localhost:8001/intelligence/rescore?tenants=nexabank&days=7"
curl -X POST "localhost:8001/intelligence/rescore?tenants=nexabank&days=30"
curl -X POST "localhost:8001/intelligence/rescore?tenants=nexabank&days=90"
```

Run all three. If you skip one, the range selector on the dashboard will have nothing to show for that range.

Confirm there is data before you open a browser:

```
curl "localhost:8001/intelligence/insights?tenants=nexabank&days=30"
```

You should get a list of findings, one per KPI. An empty list means step 4 has not finished.

Configuration

The defaults in `.env.example` are enough to run the whole demo. Copying it to `.env` is the only configuration step. These are the settings that matter.

Setting	Default	What it does
<code>VLLM_URL</code>	<code>http://host.docker.internal:11434/v1</code>	Where the narrator model lives. This is Ollama's port, 11434. The name is left over from an earlier server.
<code>INTELLIGENCE_LLM</code>	<code>1</code>	<code>1</code> turns the narrator on. <code>0</code> turns it off.
<code>CLICKHOUSE_PASSWORD</code>	<code>clickhouse</code>	Local warehouse password.

Every number and every finding is produced without the language model. Setting `INTELLIGENCE_LLM=0` still gives you a working engine end to end. Only the wording of the explanation changes.

There are no user accounts or permissions to set up. The dashboard has a persona switch at the top with four roles: CFO, Operations Manager, Risk and Compliance Officer, and Analyst. Every persona can see every metric. What changes is the answer the agent gives and what it will quote. The Operations Manager, for example, is not given revenue figures, and the answer says so instead of pretending the metric does not exist.

Never commit a secret. `.env.example` lists every variable the stack reads. Credentials are read from the environment and never from source.

Usage

Once the stack is running and the data is loaded, open these pages.

Address	What you see
http://localhost:3001/dashboard	The five KPIs, their trends, and how each one sits against its expected range
http://localhost:3001/intelligence	The evidence page: what moved, why, the recommended action, and the audit trail
http://localhost:3002	NexaBank itself, the retail bank generating the data
http://localhost:3002/admin/simulate	Plant a new anomaly and watch the engine find it

To stop everything:

```
docker compose down
```

To stop and also delete the warehouse data:

```
docker compose down -v
```

Five minute demo

1. Open <http://localhost:3001/intelligence> and pick a persona at the top.
2. In the "Ask the analyst" box, type: Why did KYC completion rate fall, and where is it concentrated? The answer names the segment, shows what each driver contributed, and proposes an action with a named owner. Open "How this answer was derived" to see which step used SQL, which used statistics, which used rules, and which used the language model.
3. Switch to Operations Manager and ask about revenue. The figure is withheld and the answer says so.
4. Open <http://localhost:3002/admin/simulate>, run the `failure_burst` template, then go back and ask about transaction failure rate. The engine finds the change you just planted, because the anomaly was written into the source data rather than drawn onto a chart.

Services and ports

Service	Port	What it is
<code>dashboard</code>	3001	The FinInsights dashboard (Next.js)
<code>nexabank-frontend</code>	3002	The NexaBank banking app (Next.js)
<code>nexabank-backend</code>	5000	NexaBank API and Simulate engine (Express, Prisma, Postgres)
<code>ingestion-api</code>	8000	Event intake, masking, deterministic event ids (FastAPI)
<code>analytics-api</code>	8001	Metric API and the intelligence layer (FastAPI)
<code>pipeline</code>	8003	Bronze, Silver and Gold transforms and the extract scheduler
<code>clickhouse</code>	8123	The analytics warehouse
<code>broker</code>	9092	Kafka, used for transport only
Ollama	11434	The narrator model, running on your machine rather than in Docker

Troubleshooting and FAQ

Problem	Cause	What to do
Answers are generic or very short	Docker cannot reach Ollama	Run <code>curl http://localhost:11434/api/tags</code> . If it fails, start Ollama and redo installation step 1.
Answers seem to be missing facts	The narrator was built without <code>num_ctx 8192</code>	Redo the <code>ollama create</code> command in installation step 1.
The dashboard shows zeros everywhere	The data is not loaded	Redo installation step 4.
"No investigation has produced an insight yet"	Nothing has been scored	Run the three <code>rescore</code> commands in installation step 4.
A page will not load	A service is still starting	Run <code>docker compose ps</code> , then <code>docker compose logs <service></code> .
A range shows no expected range	That range was never scored	Run the <code>rescore</code> command for that number of days.

Do I need a GPU? No. The model is small and runs on a normal laptop CPU.

Does any data leave my machine? No. The model runs locally through Ollama and the warehouse runs in Docker on your machine.

Can I run it without the language model? Yes. Set `INTELLIGENCE_LLM=0` . Every number and finding is still produced. Only the wording changes.

Working on the code

Everything runs through Docker. Do not use a local virtual environment, a local Node install or `npm run`. Those environments drift away from the images without warning.

The Python services (`ingestion-api` , `analytics-api` , `processor-worker`) have their source built into the image, so an edit is not live until you rebuild:

```
docker compose up -d --build analytics-api
```

The Node services (`nexabank-backend` , `nexabank-frontend` , `dashboard`) mount their source, but the file watchers do not reliably notice a change through a Windows bind mount. Restart the service before judging an edit:

```
docker compose restart nexabank-backend
```

Checking your work:

```
# Query the warehouse. The layers are bronze, silver and gold.
docker compose exec clickhouse clickhouse-client --password clickhouse \
  --query "SELECT kpi_id, count() FROM gold.kpi_daily GROUP BY kpi_id"

# Type check a TypeScript project without touching a local node_modules
docker compose exec dashboard npx tsc --noEmit

# Score the engine against the ground truth the seeder planted
docker compose --profile tools run --rm tools python scripts/run_intelligence_gates.py

# Check whether Kafka is carrying events or the ClickHouse fallback is in use
curl -s localhost:8000/health
```

Maintainers

See the contributors list on the project page: https://github.com/abhishekkumawat-47/FinInsights-Business_Intelligence/graphs/contributors